Supply-Chain Provenance & Counterfeit Detection: Research-Grade Project Analysis (Problem Statement 05)

Verdict: pursue, but narrow and reframe it. Build a clone-evident provenance ledger, not a system that claims to prove authenticity. A printed QR code can be copied perfectly. What this stack can honestly deliver is signed, hash-chained custody events, deterministic consistency rules, and scan-pattern analytics that make copying detectable after the fact and expensive to sustain. Judges will reward a team that shows this honestly with a live counterfeit-injection demo. They are much less likely to reward one that claims "unforgeable QR codes."

TL;DR

- **Pursue as a "clone-evident" platform.** Copying a code is easy. Copying it without leaving inconsistencies in a signed custody chain and in scan geography is hard. The core is a deterministic rules engine (duplicate, reuse, impossible-travel, broken-chain, quantity-mismatch) over an append-only, hash-chained PostgreSQL event log. You don't need blockchain or ML to win this.
- **Smallest honest architecture.** One Next.js app, one Node.js API, and two or three Node worker processes, all in one repo. They are connected through a transactional outbox, which writes events to PostgreSQL and then relays them to Redis Streams consumer groups. Partner events are signed with Ed25519 over RFC 8785-canonicalized JSON. QR codes carry a GS1 Digital Link-style URL with a serial and a short signed token. This satisfies every mandatory stack element without microservices or Kubernetes.
- **The fatal assumption is that scans alone can verify authenticity in real time.** If a clone is scanned before the genuine item, the first scan looks genuine. Detection is retrospective and depends on how many scans come in. Design the buyer response as risk-tiered ("verified, first check", "caution", "recalled", "unknown"), and measure precision/recall on a synthetic dataset with injected counterfeits before you demo.

1. Decision brief

Restatement. Four student developers must build, in hackathon time, a multi-tenant platform where manufacturers, logistics partners, distributors and retailers submit signed lifecycle events for product batches and units (manufacture → inspection → packaging → dispatch → receipt → sale, plus recall). The platform must enforce valid state transitions and reconcile dispatch against receipt. It must flag duplicate, reused, impossibly-travelling or orphaned identifiers, give buyers a privacy-preserving QR verification page, and trace recalls across batch lineage. The stack is mandatory: Next.js/React, Node.js, PostgreSQL, QR, digital signatures, event-driven services, Redis.

Verdict: narrow and reframe, then pursue. The problem is real and well-scoped for a demo. Three parts of the brief are naive as worded:

- "detects counterfeits" only works with enough scans;
- "protecting each partner's data" is mostly an authorization problem, not a cryptography one;
- "event-driven services" invites over-engineering.

Decisions that follow (make these in the first 90 minutes):

1. **Positioning and vertical.** Pick one vertical for realistic demo data. The recommendation is serialized pharmaceuticals or premium consumer goods (unit-level serials, high counterfeit value). Use “clone-evident, recall-ready provenance” as the pitch, not “unforgeable.”
2. **Architecture.** Use a modular monolith plus worker processes linked by a PostgreSQL outbox and Redis Streams. Don’t build microservices. Commit to this on day one so nobody spends time on Kafka, Kubernetes or blockchain.
3. **QR payload.** Encode a GS1 Digital Link-style URL with GTIN, batch, serial and a short server-verifiable token. The token stops guessed or forged serials. Scan analytics catch copied codes. Say plainly in the pitch that the token alone can’t stop photocopying.

2. Evidence, assumptions, confidence

Claim	Status	Confidence	Why it matters	Source / validation method
Global trade in counterfeits was about USD 467bn in 2021, 2.3% of global imports, up from USD 464bn in 2019. Fakes were 4.7% of EU imports (EUR 99bn)	Verified finding	High	Establishes the problem's scale for the pitch	OECD/EUIPO, *Mapping Global Trade in Fakes 2025* (7 May 2025)
A serialized QR code and its photocopy can't be told apart by a standard scan	Verified finding	High	Underpins the whole "clone-evident, not clone-proof" framing	Picard, Landry & Bolay, "Counterfeit detection with QR Codes," ACM DocEng 2021 academia researchgate
Copy-detection patterns (CDPs) can make printed codes copy-sensitive, but resistance to ML-assisted copying is an open research question	Verified finding	Medium-High	CDPs are a real-world answer, but they need print control and are out of hackathon scope	EUIPO Anti-Counterfeiting Technology Guide; Chaban et al. / Khermaza et al., IEEE WIFS 2021 (cited in arXiv 2210.05343) europa arxiv
NFC chips like NXP NTAG 424 DNA generate an AES-128 "Secure Unique NFC" (SUN) message on every tap, so replays can be detected server-side	Verified finding	High	This is the hardware path to real unclonability, and the "upgrade path" slide	NXP NTAG 424 DNA product page nxp smartlinks
Clone detection from track-and-trace data works without crypto tags, but detection and false-alarm rates trade off sharply and depend on visibility	Verified finding	Medium	Supports a rules engine with tuned thresholds and warns against overclaiming accuracy	Lehtonen, Michahelles & Fleisch, IEEE RFID 2009; Zanetti et al., "Tailing RFID Tags for Clone Detection" (ETH Zurich), which

Claim	Status	Confidence	Why it matters	Source / validation method
				reports cases such as 89% detection with 32% false alarms for some schemes udel avoine
Real verification systems are flooded with false positives at first. EU EMVS alerts were “merely under 7%” of scans at go-live in March 2019 and 0.32% by February 2021. They were about 0.08% in Q4 2024, and an EMVO update gives 0.08% again for December 2025, against a 0.05% target	Verified finding	High	Directly informs the duplicate-scan false-positive risk; most alerts came from technical/procedural errors, not fakes	tracekey on securPharm/EMVS; Bulgarian EMVS analysis (2026, ScienceDirect); EMVO “EMVO and EMVS Update” (Council of Europe); European Commission report COM(2024)274
EPCIS 2.0 / CBV 2.0 (ratified June 2022) defines what/when/where/why/how events, JSON/JSON-LD and REST bindings, and a canonical event-hash ID	Verified finding	High	Gives a credible, standard-aligned event schema and a dedup/integrity primitive	GS1 EPCIS/CBV standards; OpenEPCIS event-hash generator (CBV 2.0 §8.9) openepcis +2
GS1 Digital Link encodes GTIN, batch, serial etc. as a web URI in a QR code. Sunrise 2027 targets retail POS reading 2D codes by end-2027	Verified finding	High (concept) / Medium (exact version)	Standard-aligned QR design earns judge credibility. Vendor pages disagree on the current URI Syntax release (1.6.0 vs 1.7.0); it is also published as ISO/IEC 18975:2024	GS1 Digital Link standard; vendor explainers qrstuff +3
FSMA 204 (food traceability) compliance date is July 20, 2028. It requires Key Data Elements at Critical Tracking Events	Verified finding	High	Recall/traceability is a regulatory pull, not just a nice-to-have	Federal Register proposed rule of Aug 7, 2025 (2025-14967), to “extend the compliance date by 30 months from

Claim	Status	Confidence	Why it matters	Source / validation method
				January 20, 2026, to July 20, 2028." The November 2025 Continuing Appropriations Act then told FDA not to enforce before that date
DSCSA enhanced (package-level, interoperable electronic) requirements are in effect. Small dispensers are exempt until Nov 27, 2027 under an FDA letter of Aug 6, 2026	Verified finding	High	Pharma is a credible vertical with a live serialization mandate	FDA small-dispenser exemption letter (Aug 2026); AFS law alert afslaw aglow
Walmart's mango traceback took 6 days, 18 hours and 26 minutes on paper, and 2.2 seconds on an IBM/Hyperledger system	Verified finding	High	Recall-trace speed is a strong demo metric lfdecentralizedtrust computerworld	Computerworld report of Walmart VP Frank Yiannas's MIT "Business of Blockchain" talk: the trace "took six days, 18 hour, and 26 minutes," and in an IBM-run pilot he "was able to find the origin in 2.2 seconds"; LF Decentralized Trust case study
The speedup came mostly from digitizing and agreeing shared data, not from blockchain itself	Reasoned inference (supported by commentary)	Medium-High	Justifies PostgreSQL plus a hash chain instead of a blockchain Mother Jones	Kevin Drum, Mother Jones (May 2018): the 2.2 s "has nothing to do with blockchain and everything to do with 'cooperative partners agreeing on what information to

Claim	Status	Confidence	Why it matters	Source / validation method
				contribute'"; "garbage-in/garbage-out" oracle-problem literature (arXiv 2108.12032, 2202.13930)
Luxury consortia use private blockchains for product passports at scale. Aura reports 50+ member brands and 60M+ products (UN SDG page), and now "+80M Products" on its own site	Verified finding (self-reported)	Medium	Shows market pull for buyer-facing verification; not evidence that blockchain is necessary	UN SDG partnership page for Aura Blockchain Consortium; Aura Consortium Members page
Ed25519 signing/verification is built into Node.js `crypto` and Web Crypto, so no extra dependency is needed	Verified finding	High	Partners can sign events client-side in the browser deno +2	Node.js crypto docs
Canonical JSON is required to get repeatable signatures	Verified finding	High	Without it, signatures fail randomly across serializers	RFC 8785 (JCS) rfc-editor
PostgreSQL RLS enforces per-tenant row visibility; owners and BYPASSRLS roles skip it unless FORCE is set	Verified finding	High	The main defense against multi-tenant leaks	PostgreSQL CREATE POLICY docs dev postgresql
Redis Streams consumer groups give at-least-once delivery: entries stay pending until XACK, and XAUTOCLAIM reclaims stuck ones. Redis Pub/Sub is at-most-once	Verified finding	High	Use Streams for the anomaly pipeline and Pub/Sub only for live UI pushes. Consumers must be idempotent	redis.io XACK, XAUTOCLAIM and Pub/Sub docs redis +2
BullMQ ignores a job whose custom jobId already exists	Verified finding	High	A cheap idempotency mechanism if the	docs.bullmq.io (Job Ids, Deduplication)

Claim	Status	Confidence	Why it matters	Source / validation method
and has a deduplication option			team picks BullMQ over raw Streams	bullmq bullmq
Consumers rarely scan verification codes, so scan coverage will be sparse	Assumption	Medium	Determines whether post-sale clone detection works at all	Validate with simulation over a range of scan rates (Experiment 1)
Judges value a working end-to-end demo plus security reasoning over breadth of features	Assumption (common hackathon pattern)	Medium	Drives the MVP cut line	Check the rubric and ask organizers

3. Problem and solution fit

Target users and jobs-to-be-done

User	Current workflow	Pain	Job-to-be-done	Outcome metric
Manufacturer / brand owner	ERP plus serialization line; EPCIS or CSV exports to partners; brand-protection team works manually on reports	No view after dispatch; grey-market diversion; clones surface only through complaints	"Know where every batch is and spot fakes before customers do"	Time-to-detect a clone; % of units with complete chain
Logistics partner	TMS/WMS, paper or EDI proof-of-delivery	Disputes over shortages; blamed for losses	"Prove what I received and handed over"	Disputes resolved; reconciliation lag
Distributor / wholesaler	Receives ASNs, verifies on receipt (pharma: DSCSA)	Mismatched quantities; suspect product investigations	"Accept only legitimate stock, fast"	Receipt-to-reconciled time; mismatch rate
Retailer / dispenser	Scans at receiving and POS	Saleable-returns verification; recall notices too broad	"Pull only affected stock"	Recall scope precision (units pulled ÷ units actually affected)
Buyer / consumer	Checks packaging, maybe scans a QR	Can't tell genuine from fake; marketing QRs build no trust	"Is this real, and is it safe?"	Verification completion rate; clarity of outcome
Recall coordinator / regulator	Requests one-up/one-back records, spreadsheets	Days to reconstruct the chain	"Who got the affected lots, now?"	Time to produce an impact report (target under 5 s in demo vs days on paper)

What must be true for it to work:

1. Partners submit events promptly and honestly, using keys they control. Signatures prove *who* said something, not that it's *true*, so the oracle problem remains.
2. Enough downstream scans exist (retail receipt, POS sale, some consumer scans) for duplicates to show up.
3. Each identifier is unique and commissioned exactly once, by the brand owner.
4. Visibility rules are enforced in the database, not just in the UI.
5. Buyers get a simple, honest answer. False "fake" alarms on genuine products erode trust faster than missed clones do, as the early EMVS experience showed.

4. Landscape and shared patterns

Approach	Examples	Pattern	Benefit	Limitation
GS1 identification and event standards	GTIN/SGTIN, SSCC, GLN, EPCIS 2.0 + CBV 2.0, GS1 Digital Link	Standard identifiers; events structured as what/when/where/why/how; bizStep and disposition vocabularies; canonical event hash	Interoperable, regulator-familiar, gives an off-the-shelf event schema	Defines data, not trust or anomaly logic. Full EPCIS conformance is heavy for a hackathon anglera openepcis
Regulatory traceability rules	DSCSA (US pharma), EU FMD/EMVS, FSMA 204 (food)	Mandated serialization, verification and decommissioning, or KDEs at CTEs; time-bound record production	Creates real demand. EMVS shows a central verify-and-decommission repository working at EU scale	Huge false-alert burden at go-live; designed around regulated actors, not consumers pharmait gandlscientific
Serialization / L4–L5 vendors	TraceLink, OpenEPCIS repositories, iFoodDS	Hosted EPCIS repositories, partner onboarding, compliance reporting	Mature integration	Expensive and compliance-focused; anomaly analytics are usually thin
Blockchain provenance	IBM Food Trust (Hyperledger Fabric), VeChain, OriginTrail DKG, Aura Consortium	Shared ledger or hash-anchoring; permissioned access; consumer-facing passports	Multi-party tamper evidence without a single trusted operator; strong marketing pull in luxury	Garbage in, garbage out: the ledger can't verify physical truth. Slow governance; hard to justify when one operator exists lfdecentralizedtrust Mother Jones ciodive +2
Physical authentication	QR + copy-detection patterns, NFC NTAG 424 DNA (SUN/CMAC per tap),	Make the physical carrier hard to copy, or produce a per-read cryptographic proof	Only route to genuine clone *prevention*	Costs per unit; needs print or chip control; CDPs face ML-copying research europa tp-rfid

Approach	Examples	Pattern	Benefit	Limitation
	holograms, tamper seals			
Track-and-trace analytics research	Lehtonen et al. (clone detection from incomplete RFID traces), Zanetti et al. ("tailing"), anomaly-detection literature	Flag impossible transitions, duplicate concurrent presence, deviations from the supply-chain model	Works with cheap, copyable tags; matches this brief	Probabilistic; trade-off between detection and false alarms; sensitive to missing reads udel researchgate

Shared pattern across all of them: *identity + signed events + a consistency model + a verification interface*. Every serious system separates identification (what), evidence (who said what, when, signed), and inference (does this story make physical sense). The hackathon project should copy that separation.

5. Non-obvious insights and differentiation

Key insights

1. **A QR code is a pointer, not a proof.** A photocopy is identical (Picard et al.). academia Counterfeits are caught by *inconsistency*: the same serial in two places, a serial that was already sold or decommissioned, a serial outside its authorized distribution route, or a chain with missing custody. Make "clone-evident" the headline concept.
2. **How to make codes clone-*detectable*, ordered from cheapest to strongest:**
   - *Random, non-sequential serials.* At least 64 bits of entropy, so serials can't be guessed or enumerated.
   - *Signed or HMAC tokens in the QR URL,* e.g. `t = base64url(HMAC-SHA256(k_brand, gtin|batch|serial))[:10]`, or a truncated Ed25519 signature. These stop forged *new* serials, not copies of real ones.
   - *Server-side state.* Each unit has a lifecycle (commissioned → … → sold → consumer-checked), and a consumer scan of a unit not yet sold, or already "claimed," is a signal.
   - *Scan-count and diversity thresholds,* counting distinct devices and regions rather than raw scans.
   - *Geo-velocity checks.*
   - *Optional "claim" step* (scratch-off PIN or proof of purchase) to bind first ownership.
   - *Hardware upgrade path:* NTAG 424 DNA, which produces a per-tap CMAC with a monotonic counter, so a replayed URL is detectable. smartlinks

3. **A hash-chained, append-only PostgreSQL log can replace blockchain here.** With a single operator, the threats are an insider rewriting history and a partner disowning its own events. Per-event partner signatures defeat repudiation. A hash chain (`hash_n = SHA-256(hash_{n-1} || canonical_event || signature)`) plus revoked UPDATE/DELETE grants and a trigger make tampering evident. Optionally publish a daily chain head, for example to a public Git repo or by emailing it to partners, as an external anchor. The Walmart 2.2-second traceback came mostly from digitized, shared data, which this design also has.
4. **Partner data protection is mostly an authorization problem.** The privacy gains come from RLS, field-level projection, and per-audience views (owner / custodian / downstream partner / public buyer). Buyer pages should show *conclusions* ("sold by an authorized retailer in your region; chain complete"), not partner names, quantities or prices.
5. **Event-driven correctness comes down to idempotency and ordering.** Redis Streams delivers at least once, so every consumer must deduplicate on `event_id`. redis Partner clocks lie, so order by server `recorded_at` for processing and use the partner's `event_time` only for business rules, within a skew tolerance. Late events must re-trigger evaluation of downstream rules.
6. **False positives are the product-killer, not false negatives.** The EU system's early alert flood came from data and process errors. raps Every rule therefore needs a severity tier, an explanation string, and a "resolve as benign" workflow.

Differentiation ladder

Tier	Features
Table stakes (every team will have these)	QR generation and lookup; CRUD for batches and events; a role-based dashboard; a basic "duplicate scan" counter
Meaningful differentiation	Ed25519-signed events with canonical JSON and a visible verification badge; a hash-chain integrity check with a tamper demo; a deterministic state machine with rejection reasons; dispatch/receipt reconciliation handling partial shipments; an explainable risk score with geo-velocity; RLS-enforced views per audience; a recursive-CTE recall impact report in under 1 second; precision/recall measured on an injected-counterfeit dataset
Potential moat (pitch only; out of scope)	A network effect from cross-partner scan data; a learned per-SKU "normal route" model; NFC SUN integration; EPCIS 2.0 import/export to interoperate with existing serialization systems; DPP/GS1 Digital Link resolver compatibility

6. Blind spots and risk register (ranked)

#	Risk	Likelihood	Impact	Early signal	Mitigation / test
1	**FATAL ASSUMPTION: scans can authenticate in real time.** A clone scanned before the genuine unit looks genuine; sparse consumer scans hide clones	High	High	Simulation shows recall under 50% at realistic scan rates	Reframe as risk-tiered and retrospective. Add a "claim on first check" step. Measure detection versus scan rate (Experiment 1) and show the curve to judges
2	Scope creep for 4 people (6 roles × 7 event types × 3 modules)	High	High	No end-to-end flow by the 30% mark	Hard MVP cut line (§8); walking skeleton first; feature freeze at 75% of time
3	False positives on duplicate scans (family members, shop staff re-scanning, page refreshes, link previews)	High	Medium-High	Genuine demo flows trigger "caution"	Count distinct device fingerprints, not raw scans; ignore repeats from the same device within 24 h; skip link-preview user agents; require two signals for "high risk"
4	Impossible-travel errors from unreliable timestamps or geolocation (IP geolocation at city level, VPNs, clock skew, timezone bugs)	Medium-High	Medium	Flags between nearby cities	Store all times in UTC; use server receive time for consumer scans; subtract location-accuracy radii from distance; add a skew allowance; treat a VPN or datacenter ASN as "unknown location", not "far away"

#	Risk	Likelihood	Impact	Early signal	Mitigation / test
5	Multi-tenant visibility leak (a partner sees a competitor's routes, quantities or prices)	Medium	High	Any API returns another tenant's rows in tests	PostgreSQL RLS with FORCE; app role without BYPASSRLS; `SET LOCAL app.partner_id` per transaction; automated cross-tenant leak tests in CI; separate public projection view for buyers dev
6	Key management and signature trust (who vouches that a key belongs to a partner; lost or stolen keys; demo keys hard-coded)	Medium	High	All partners share one key in seed data	Register keys at onboarding by admin approval (key fingerprint shown out-of-band); `partner_keys` with valid_from/valid_to/revoked_at; reject events signed by revoked keys after `revoked_at`; private keys stay in browser or CLI, never on the server
7	Quantity reconciliation edge cases: partial shipments, multi-drop routes, split/merged batches, rework, returns, damaged goods	High	Medium	Reconciliation fires on legitimate partial deliveries	Model shipments with lines and expected-vs-received per line; aggregation and transformation events for split/merge; tolerances per unit of measure; a "reason code" on mismatched receipts
8	Event ordering and duplicate delivery in workers	Medium	Medium	Anomalies created twice; receipt processed before dispatch	Idempotency key per event (unique index plus Redis SET NX); outbox pattern; "pending predecessor" state that re-evaluates when the dispatch arrives
9	Demo-data realism (toy data makes detection look trivial)	High	Medium	Judges ask "how would this behave with real noise?"	Seeded generator with realistic routes, delays, partial shipments, benign duplicate scans and injected counterfeits; publish precision/recall
10	Stack-rule penalty, from forgetting a	Low-Medium	High (−10	Redis used only as a cache; no	Stack-compliance checklist slide mapping each required tech to a

#	Risk	Likelihood	Impact	Early signal	Mitigation / test
	mandated element or making it superficial		marks)	signatures	concrete function
11	Overclaiming in the pitch ("unforgeable", "blockchain-grade")	Medium	Medium	Judge challenge on copied QR codes	Lead the pitch with the cloning problem yourselves

Single fatal assumption: that QR scans can tell a buyer *at the moment of scanning* whether an item is genuine. They can't do that reliably. If the team builds the pitch on it, a knowledgeable judge will dismantle it with one photocopy. Reframe early.

7. Validation plan (priority order)

#	Hypothesis	Method	Data	Success / failure threshold	Owner	Time	Next decision
1	Rules detect injected clones with useful precision at plausible scan rates	Synthetic generator: 3 manufacturers, 20 batches, 2,000 units, 8 nodes, realistic transit times; inject 3–5% cloned serials, plus benign noise (repeat scans, partial shipments, 2% clock skew); sweep consumer scan rate 5–60%	Seeded generator (Person 4)	Success: precision ≥ 0.90 and recall ≥ 0.70 at a 30% scan rate; high-risk false-positive rate under 1% of genuine units. Failure: precision under 0.75	P4 (data) + P2 (rules)	3–4 h	Tune thresholds; if recall stays low, add the "claim" step and say so in the pitch
2	The signature flow is correct and usable	Browser generates an Ed25519 key with Web Crypto; the server verifies over JCS-canonical bytes; test tampered payload, wrong key, revoked key, replayed nonce, reordered JSON keys	Unit tests	100% of tamper cases rejected; reordered keys still verify; signing round-trip under 50 ms	P3	2 h	If browser Ed25519 is unreliable on judges' machines, fall back to a Node CLI signer or ECDSA P-256
3	Concurrent duplicate scans don't double-count or race	Fire 200 parallel scans of one serial from 10 simulated devices with autocannon/k6; Redis INCR plus SET NX dedupe	Load script	Counter equals the expected distinct count; exactly one anomaly created; p95 verify latency under 300 ms locally	P2	1.5 h	If races appear, move the anomaly upsert to a PostgreSQL unique constraint

#	Hypothesis	Method	Data	Success / failure threshold	Owner	Time	Next decision
4	RLS prevents cross-tenant reads	Automated tests: partner A queries partner B's shipments, events and prices via API and raw SQL with the app role	Seed tenants	Zero rows leaked across 20 test queries	P1	1.5 h	If it fails, block the release
5	Recall tracing is fast and complete	Recursive CTE over lineage (5 levels, 2,000 units)	Seed data	Impact report under 1 s; matches brute-force ground truth exactly	P1	1 h	Add indexes or a materialized path if slow
6	Buyers understand the verification result	Hallway test with 5 people: show 4 result states and ask "what would you do?"	Paper or Figma mock	At least 4/5 interpret each state correctly	P4	45 min	Rewrite copy and iconography
7	The outbox → Streams → worker pipeline survives a worker crash	Kill the worker mid-batch; restart; check that XAUTOCLAIM reprocesses and no duplicate anomalies appear	Integration test	No lost events; no duplicates	P3	1 h	If flaky, switch to BullMQ with jobId equal to event_id mintlify

8. Product and technical direction

Lean first release

In scope (MVP cut line, which must work end to end):

- Partner onboarding with roles (manufacturer, logistics, distributor, retailer, admin) and Ed25519 public-key registration
- Batch and unit creation; QR generation (GS1 Digital Link-style URL + token); printable label sheet
- Signed event submission for the 6 lifecycle steps plus recall; state-machine validation with clear rejection reasons
- Hash-chained append-only event log plus an "integrity check" button

- Dispatch/receipt reconciliation (quantity and route mismatch) including partial shipments
- Rules engine: duplicate/reused code, impossible travel, broken chain, quantity mismatch, scan-diversity threshold, with risk scores and explanations
- Public buyer verification page with 4 result states and no confidential fields
- Recall: mark a batch recalled, trace downstream impact, generate an impact report (HTML/PDF/CSV)
- Live anomaly feed on the brand dashboard (Redis Pub/Sub → SSE)

Stretch (only after the cut line is green): EPCIS 2.0 JSON-LD export of events; map visualization of routes; batch split/merge UI; anomaly "resolve as benign" workflow.

Explicitly out of scope: blockchain, microservices or Kubernetes, ML models in production, real GS1 company-prefix licensing, NFC hardware, copy-detection-pattern printing, payments, ERP integrations, mobile native apps.

Architecture: modular monolith plus event-driven workers

```
[Next.js web (partner portal + buyer /verify page)]
        | HTTPS (signed event envelopes)
[Node.js API (Fastify/Express or Next route handlers)]
   ├─ validate schema + signature + state transition
   ├─ BEGIN; INSERT event (hash-chained) + INSERT outbox; COMMIT
   └─ /verify: Redis rate-limit → PG lookup → Redis counters → response
[Outbox relay] → Redis Stream "events" (XADD)
[Workers via consumer groups (XREADGROUP/XACK)]
   ├─ anomaly-worker (rules engine → anomalies table)
   ├─ reconciliation-worker (dispatch↔receipt matching)
   └─ projection/notify-worker (public projection, Pub/Sub → SSE dashboard)
[PostgreSQL] system of record (RLS)    [Redis] streams, counters, cache, rate limits, idempotency
```

This honestly satisfies "event-driven services": independent processes communicate only through events, and the API never calls the rules engine synchronously. It stays one repo, one `docker-compose.yml`, and one language. Splitting services out later is a deployment change, not a rewrite.

Key data flows:

1. **Event submission:** client builds payload → JCS canonicalize → Ed25519 sign → POST envelope → API verifies signature against the active partner key, checks nonce and timestamp window, checks idempotency key, validates the transition → one transaction writes `events` (with prev_hash/hash) and `outbox` → relay XADDs → workers process and XACK.
2. **Buyer verification:** scan URL → `/verify` → rate-limit by IP → token check (constant-time) → load unit state and public projection → record scan (PG insert plus Redis counters) → publish scan event → respond with result state. Geo-velocity is evaluated synchronously from the Redis cache so the result can include "caution".

3. **Recall:** admin or manufacturer issues a signed RECALL event on a batch → worker runs the lineage CTE → `recall_impacts` rows → notifications to affected custodians → buyer page shows "RECALLED" immediately (cache invalidated).

Event-state machine

From state	Allowed events → new state	Actor role (must be current custodian unless noted)
(none)	MANUFACTURE → `manufactured`	Manufacturer (brand owner of the GTIN)
manufactured	INSPECT(pass) → `inspected`; INSPECT(fail) → `quarantined`	Manufacturer / QA
inspected	PACK → `packed` (may aggregate units into case/SSCC)	Manufacturer
packed / received	DISPATCH(shipment_id, to_partner, qty) → `in_transit`	Current custodian
in_transit	RECEIVE(shipment_id, qty) → `received` (custody moves to receiver)	The shipment's addressee only
received	DISPATCH → `in_transit`; SELL → `sold`	Distributor / retailer
sold	CONSUMER_CHECK (non-state-changing); RETURN → `returned`	Public / retailer
any non-terminal	RECALL → `recalled` (overlay flag, preserves location)	Manufacturer / regulator role
quarantined, recalled	DESTROY → `destroyed` (terminal)	Custodian

Rejected transitions return `409` with a machine code (e.g. `ILLEGAL_TRANSITION: sold→in_transit`), and the attempt is logged as a low-severity anomaly. Repeated illegal attempts on one serial are themselves a signal.

PostgreSQL data model sketch

- `partners(id, name, role, country, status, created_at)`
- `partner_keys(id, partner_id, alg='Ed25519', public_key, fingerprint, valid_from, valid_to, revoked_at)`

- `products(gtin, brand_partner_id, name, public_attributes jsonb)`
- `batches(id, gtin, lot, mfg_date, expiry, owner_partner_id, status, recalled_at)`
- `batch_lineage(parent_batch_id, child_batch_id, relation[split|merge|transform], event_id)`
- `units(id, batch_id, serial, token_hash, state, current_custodian_id, sold_at, sold_region, first_consumer_scan_at)` with `UNIQUE(batch_id, serial)` and a global `UNIQUE(gtin, serial)`
- `aggregations(parent_sscc, child_unit_id, event_id, active)`
- `shipments(id, from_partner, to_partner, expected_route jsonb, dispatched_at, received_at, status)`; `shipment_lines(shipment_id, batch_id, qty_dispatched, qty_received, unit_ids uuid[])`
- `events(id, seq bigserial, event_type, subject_type, subject_id, partner_id, key_id, event_time, recorded_at, payload jsonb, canonical_hash, signature, nonce, prev_hash, chain_hash, idempotency_key UNIQUE)` with UPDATE/DELETE revoked and a trigger that computes `chain_hash`
- `scans(id, unit_id, scanned_at, ip_hash, device_fp_hash, geo_lat, geo_lon, geo_accuracy_km, asn_type, user_agent_class, context[consumer|retail|logistics])`
- `anomalies(id, rule_code, severity, score, subject_id, evidence jsonb, explanation, status[open|benign|confirmed], dedupe_key UNIQUE)`
- `reconciliations(shipment_id, line_id, delta_qty, route_ok, status, reason_code)`
- `recall_impacts(recall_id, batch_id, partner_id, units_count, units_sold, units_in_stock)`
- `outbox(id, event_id, stream, published_at)`

RLS policies: partners see rows where they are owner, sender, receiver or custodian. Downstream partners see upstream events only as a redacted projection view. The `public_unit_view` exposes only product name, batch, expiry, recall flag, chain-complete boolean, sale region (coarse) and scan-count bucket.

Signed event design

- **Algorithm: Ed25519.** It is deterministic (no nonce-reuse foot-gun, unlike ECDSA), with 64-byte signatures, fast verification, and native support in Node `crypto` (`sign(null, data, key)`) and Web Crypto. mojoauth github Use ECDSA P-256 only if hardware-backed keys (WebAuthn/HSM/cloud KMS) are required. That trigger is unlikely in a hackathon but worth a sentence in the pitch.
- **Canonicalization: RFC 8785 JCS.** Sorted keys, ECMAScript number serialization, I-JSON subset. rfc-editor Sign `JCS(payload)`, where the payload includes `{event_type, subject, event_time, partner_id, key_id, nonce, prev_event_ref?, data}`.
- **Envelope:** `{payload, sig: base64url, alg: "Ed25519", kid}`.
- **Key registration:** at onboarding the partner portal generates the key pair in the browser. The private key is exported encrypted, or kept in IndexedDB for the demo. The public key and fingerprint go to the server, and an admin approves by comparing the fingerprint read aloud or sent out-of-band. Rotation adds a new key with `valid_from`; revocation sets `revoked_at`.

- **Replay protection:** 128-bit random `nonce`, unique per partner (Redis `SET nonce:{partner}:{nonce} NX EX 86400` plus a PG unique constraint); `event_time` within ±10 min of server time for live submissions; batched offline uploads are allowed with a flag and stricter review; `idempotency_key = sha256(kid|nonce)`.
- **Verification UI:** each event in the timeline shows a green "signed by Partner X (key ab12…)" badge and a chain-integrity status.

Redis uses (each one concrete; this matters for the stack-compliance score)

Use	Mechanism
Event bus	Streams `events` with consumer groups `anomaly`, `recon`, `projection`; XACK after commit; XAUTOCLAIM for stuck messages (idle over 60 s) oneuptime redis
Live dashboard	Pub/Sub channel `anomalies:{brand}` → Node SSE endpoint (at-most-once is fine here) redis
Idempotency and replay	`SET idem:{key} NX EX 86400`
Rate limiting	Sliding window on `/verify` per IP (e.g., 30/min) and per serial (e.g., 10/min)
Scan counters	`INCR scans:{unit}`, `PFADD devices:{unit} {fp}` (HyperLogLog for distinct devices), `PFADD regions:{unit} {region}`
Geo-velocity cache	`HSET lastscan:{unit} lat lon ts acc` for O(1) comparison
Hot cache	Public unit projection (`GET pub:{gtin}:{serial}`), invalidated on recall

QR design

Recommendation: a **GS1 Digital Link-style URL with a short token**, e.g. `https://verify.example.com/01/09506000134352/10/LOT42/21/7KQ9XW2M4RTA?t=Hq3vB8sZ1x`.

- *Opaque ID only:* simplest and private, but no standards credibility.
- *Fully signed self-contained token (JWS in the QR):* verifiable offline, but makes a dense QR and is still copyable.
- *GS1 DL + HMAC token:* standard-aligned (scanning-ready for POS under Sunrise 2027), works on any phone camera, and the token blocks enumeration of serials. **Choose this.** qr-verse qr-verse

Use a demo GTIN prefix and say "GS1-format, unlicensed demo prefix." Use error-correction level M or Q.

Anomaly rules engine (deterministic; each emits severity, score and explanation)

Rule	Formula / condition	Default threshold	Severity
R1 Unknown or forged code	Serial not found, or token ≠ HMAC(k, gtin|batch|serial)	any	High (shown as "not recognized")
R2 Reused / duplicate commissioning	Second MANUFACTURE for the same (gtin, serial), or a serial inside two active aggregations	any	Critical
R3 State-inconsistent scan	Consumer scan when state ∈ {manufactured…in_transit} (not yet sold), or state = destroyed	any	High
R4 Impossible travel	v = max(0, haversine(p1,p2) − acc1 − acc2) / max(Δt + skew, ε); flag if v > V_max	V_max = 900 km/h (consumer, air-plausible); 120 km/h for road-only logistics legs; skew = 120 s; skip if either location is VPN/datacenter	High
R5 Scan diversity after sale	distinct_devices_30d ≥ D or distinct_regions_30d ≥ 2 (region ≠ sold_region)	D = 4 (repeats from the same device within 24 h ignored)	Medium → High if combined with R4
R6 Broken chain / custody gap	Event submitted by a partner ≠ current_custodian; RECEIVE without a matching DISPATCH; a gap where unit location jumps between nodes with no shipment	any	High
R7 Quantity mismatch	Δ = qty_dispatched − qty_received per shipment line; flag if |Δ| > tolerance; for serialized units compare sets (missing, extra, foreign serials)	tolerance 0 for serialized, 0.5% for bulk by weight; partial receipts open for 48 h before flagging	Medium (shortage) / High (foreign serials)
R8 Route deviation	Receiver ∉ authorized distribution list for the batch, or the receipt location is more than X km from the expected destination GLN	X = 50 km	Medium

Rule	Formula / condition	Default threshold	Severity
R9 Post-recall movement	DISPATCH or SELL of a recalled batch	any	Critical

Risk score: `score = 1 − Π(1 − w_i)` over triggered rules (weights 0.2–0.9), mapped to buyer states: under 0.3 → "Verified", 0.3–0.7 → "Caution", over 0.7 or R1/R2 → "Not verified". Recall overrides everything.

Buyer-facing verification response

- **Verified** ✓: "Genuine product record found. Made by {Brand} · Batch {LOT} · Expires {date} · Sold by an authorized retailer in {region}. This code has been checked {1 / 2–5 / many} times."
- **Caution** ⚠: "This code has been checked in multiple places. Keep proof of purchase and contact {Brand}." Include a report button.
- **Not verified** ✗: "We can't confirm this product." Include a report button.
- **Recalled** ⛔: the recall reason (public text) and instructions.

Never shown: logistics partner names, routes, quantities, prices, other partners' identities, exact scan locations. Responses are cached per serial with short TTLs and rate-limited.

Recall tracing

sql

```sql
WITH RECURSIVE affected AS (
  SELECT id AS batch_id, 0 AS depth FROM batches WHERE id = $recalled
  UNION
  SELECT bl.child_batch_id, a.depth + 1
  FROM batch_lineage bl JOIN affected a ON bl.parent_batch_id = a.batch_id
  WHERE a.depth < 10
)
SELECT p.name, sl.batch_id, SUM(sl.qty_received) AS units_received,
       COUNT(u.id) FILTER (WHERE u.state='sold') AS units_sold,
       COUNT(u.id) FILTER (WHERE u.state='received') AS units_in_stock
FROM affected a
JOIN shipment_lines sl ON sl.batch_id = a.batch_id
JOIN shipments s ON s.id = sl.shipment_id
JOIN partners p ON p.id = s.to_partner
LEFT JOIN units u ON u.batch_id = a.batch_id AND u.current_custodian_id = p.id
GROUP BY p.name, sl.batch_id;
```

The impact report shows affected batches (including descendants from splits/merges), partners holding stock, units sold vs in stock, and regions. It is generated as a signed PDF/CSV, in under 1 s on demo data.

Use UNION (not UNION ALL) plus a depth cap to guard against cycles.

Non-functional requirements (demo-grade targets)

- `/verify` p95 under 300 ms locally; event ingest p95 under 500 ms.
- Zero cross-tenant leaks in the automated test suite.
- 100% of event mutations signed; chain-integrity check over all events in under 2 s for 10k events.
- Worker crash recovery with no lost or duplicated anomalies.
- All times UTC; idempotent endpoints; seed script rebuilds the whole demo in under 2 minutes; one-command `docker compose up`.

Technology decision table

Layer	Recommendation	Why now	Trade-off	Alternative / trigger to switch
Frontend	Next.js (App Router) + React + Tailwind + shadcn/ui	Mandatory; fast UI; SSR verify page loads fast on phones	App Router learning curve for juniors	Pages Router if the team knows it better
API	Node.js (Fastify or Express) in the same monorepo; Zod schemas	Mandatory; one language; Zod shared between client and server	Separate process to run	Next route handlers only if a member lacks backend experience
DB	PostgreSQL 16 + RLS + recursive CTEs + triggers	Mandatory; RLS gives real isolation; CTEs handle recall	RLS debugging is subtle	Neo4j only if lineage depth or graph queries become the core product
Events	Transactional outbox → Redis Streams consumer groups	Honest event-driven design with at-least-once delivery and replay	You write the relay and the idempotency logic	BullMQ (jobId dedupe) if Streams code proves fragile; Kafka/NATS only for multi-team scale `bullmq`
Cache / limits	Redis (counters, HLL, rate limiting, idempotency)	Mandatory; O(1) scan analytics	State is split across two stores	—
Signatures	Ed25519 via Node crypto / Web Crypto; RFC 8785 canonicalize (`canonicalize` npm)	Built in, deterministic, small	Browser key storage is weak	ECDSA P-256 + WebAuthn/KMS for hardware-backed keys
QR	`qrcode` npm (server), `@zxing/browser` or `html5-qrcode` (scanner)	Mature, client-side	Camera permissions on some laptops	Native camera app opening the URL
Integrity	Hash chain in PG + daily published head	Tamper evidence without blockchain	Single-operator trust remains	Public-chain anchoring (one hash per day) if judges value decentralization; full ledger

Layer	Recommendation	Why now	Trade-off	Alternative / trigger to switch
				only for a multi-operator mandate
Infra	docker-compose; deploy to Vercel (web) + Railway/Render/Fly (API, workers, PG, Redis)	Reproducible demo	Free tiers sleep	Run locally on the demo laptop as the fallback

Where extra tech could add scoring value, only after the mandatory stack works: EPCIS 2.0 JSON-LD export (standards credibility); a map (MapLibre) for impossible-travel visuals; a once-a-day public timestamp anchor of the chain head; an NFC NTAG 424 DNA demo if someone owns a tag; an LLM that writes a plain-English summary of an anomaly cluster or recall report. That last one is optional and presentational only. It must never make the decision.

Is ML justified? Not for the MVP. The requirements are deterministic rules on sparse, labeled-by-construction data, and judges can audit rules. If time remains, run an **Isolation Forest** over per-unit scan features (distinct devices, regions, inter-scan intervals, max velocity, time since sale) as a *secondary* score. Compare it with the rules baseline on the same synthetic test set using precision/recall/PR-AUC at a fixed false-positive rate, and report honestly whether it adds lift. With synthetic data, any "AI" claim is about the method, not real-world accuracy, so say so.

9. Team plan (4 people; skills assumed mixed)

Person	Primary role	Accountability	First deliverable (≈ first 15% of time)	Dependencies / collaboration boundary
P1: Data & platform lead (strongest backend/SQL)	Schema, RLS, state machine, recall CTE, hash chain	Data integrity, tenant isolation, recall report	Migrations + seed tenants + RLS policies + `events` table with chain trigger	Owns the DB contract; others call through the repository layer only. Reviews every migration
P2: Events & detection	Outbox relay, Redis Streams workers, rules engine, scan counters, reconciliation	Detection precision/recall numbers	Walking skeleton: event → outbox → stream → worker logs it	Consumes P1's schema; exposes `anomalies` for P4's UI; pairs with P4 on the generator
P3: Security & API	Signature envelope, JCS, key onboarding, nonce/idempotency, `/verify` endpoint, QR/token generation	"Every event signed; every tamper rejected"	Sign-in-browser → verify-in-Node round-trip test passing	Owns the API contract (OpenAPI/Zod); P4 builds UI against mocks of it from hour 1
P4: Product, UX, demo & QA (front-end/design-leaning)	Next.js portal + buyer page, synthetic data generator, test scripts, pitch	Demo storyline, usability, final deck and video backup	Clickable mock of 4 screens (partner event form, timeline, anomaly feed, buyer verify) + demo script v0	Builds against mocked API; owns the "feature freeze" call and the rehearsal schedule

Substitutions: if nobody is strong in SQL, P1 uses Prisma or Drizzle for CRUD but writes RLS and the CTE in raw SQL with help from P2. If someone is very junior, give them the data generator and test scripts. These are high-value, low-coupling work.

Phase plan (as % of total hackathon time; adapts to 24–48 h):

Phase	Share	Content	Exit criterion
0. Align and validate	8%	Decisions 1–3, rubric check, API contract, schema v1, demo script v0, mocks	Contract frozen; repo and compose running
1. Walking skeleton	22%	One batch → signed MANUFACTURE → DISPATCH → RECEIVE → SELL → buyer scan, end to end through the outbox/stream	The "golden path" works on one laptop
Experiment buffer A	10%	Spike risky bits: browser Ed25519, RLS context, XAUTOCLAIM; allowed to fail and fall back	Each spike kept or replaced by its fallback
2. Detection and reconciliation	20%	Rules R1–R8, reconciliation, generator, Experiment 1 metrics	Precision/recall table produced
3. Recall, privacy, polish	15%	Recall CTE + report, public projection, anomaly feed, integrity check	All MVP cut-line items green
Feature freeze + buffer B	10%	Bug fixing only; leak tests; load test; seed reset script	No P0 bugs; demo reset under 2 min
4. Demo prep	15%	Two full rehearsals, backup video, slides with metrics, Q&A drill on "can't I just copy the QR?"	Timed run ≤ the slot minus 30 s

Coverage gaps to watch: nobody owns DevOps (assign P3 a 1-hour deploy spike early); accessibility and mobile layout of the buyer page (P4); someone must sleep, so stagger rest so that P1/P2 and P3/P4 aren't both offline.

What judges are likely to reward, and demo storylines

Judges reward a coherent story, a live end-to-end working flow, visible technical depth (signatures, chain, events), honest limits, and measured results. Feature checklists matter less. (This is a reasoned inference from common hackathon rubrics; check the actual rubric.)

Demo storyline (≈5 min):

1. **Hook (30 s):** "OECD and EUIPO valued counterfeit trade at about USD 467bn in 2021, 2.3% of global imports. Here's a genuine bottle with a QR code, and here's a photocopy of that code. A normal scanner can't tell them apart. Neither can we, at the moment of the scan. Watch what happens next."
2. **Provenance (60 s):** the manufacturer signs MANUFACTURE → INSPECT → PACK; the logistics partner dispatches; the distributor receives *8 of 10 cases*. Reconciliation flags the 2-case shortage live on the dashboard via Redis → SSE.
3. **Counterfeit injection (90 s):** a teammate scans the genuine code "in Berlin" (sold there); another scans the photocopy 20 minutes later with a spoofed location "in Lagos". The impossible-travel and

region-diversity rules fire; the buyer page turns **Caution**; the anomaly card explains the math ("4,700 km in 20 min = 14,100 km/h > 900 km/h").

4. **Tamper (45 s):** edit an event row directly in psql → click "Verify chain" → the break is pinpointed at event #1,842; submit an event with a forged signature → rejected with 401 `BAD_SIGNATURE`.
5. **Recall (45 s):** recall batch LOT42 → the recursive trace includes a split child batch → impact report in under 1 s: 3 partners, 1,140 units in stock, 260 sold, regions listed. The buyer page for any affected unit now shows **Recalled**. Contrast with Walmart's paper traceback of 6 days 18 h 26 min. computerworld Mother Jones
6. **Proof (30 s):** a slide with precision/recall from 2,000 synthetic units and the honest limit: "First-scan clones can't be caught instantly; here's our detection-vs-scan-rate curve and the NFC upgrade path."

Risk-aware MVP cut line: if time collapses, keep, in this order: (1) signed events + state machine, (2) buyer verify with R1/R3/R4, (3) recall CTE, (4) reconciliation, (5) hash-chain tamper demo. Drop the map, PDF export, split/merge UI, and ML first.

Caveats

- Every detection metric in this plan comes from synthetic data. Real-world false-positive rates will be higher, as EMVS's early alert rate of almost 7% showed. tracekey Present the numbers as validation of the method, not as field accuracy.
- Signatures prove who submitted an event, not that it's physically true. A dishonest partner can still sign false data (the oracle problem). arxiv arxiv Reconciliation across independent parties is the mitigation.
- Regulatory dates (DSCSA, FSMA 204, Sunrise 2027) were checked as of October 2026 and have moved before; re-verify before quoting them on stage.
- IBM Food Trust's current commercial status wasn't confirmed (its latest visible move is the 2023 iFoodDS Trace Exchange partnership). food-safety Don't name it as a live competitor without checking.

10. Immediate next moves

1. **In the first hour, lock the three decisions** (vertical, modular monolith + Streams, GS1-DL + token QR) and freeze the API/event contract so all four people can work in parallel against mocks.
2. **Build the synthetic data generator with counterfeit injection on day one.** It feeds the demo, the precision/recall slide, and every test, and it surfaces weak premises (scan sparsity, false positives) before the pitch does.
3. **Get the signed golden path working end to end** (browser Ed25519 → API verify → hash-chained insert → outbox → Redis Stream → worker → buyer page) before writing any second feature.

The single question that would most change this recommendation: *what does the judging rubric weight most: technical depth and security, or business viability and UX?* If depth dominates, invest the buffer in tamper and replay demos, EPCIS export and measured detection curves. If viability dominates, invest it in the buyer experience, the recall report and a sharper single-vertical story (e.g., DSCSA-style pharma), and cut the integrity extras.